

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# --- STEP 1: LOAD ALL FILES ---
files = {
    'campaign_household': '/content/drive/MyDrive/Capstone_Project/RQ1_
    'store_performance': '/content/drive/MyDrive/Capstone_Project/RQ1_
    'category_store': '/content/drive/MyDrive/Capstone_Project/RQ2_ca
    'discount_dependency': '/content/drive/MyDrive/Capstone_Project/R
    'demographics': '/content/drive/MyDrive/Capstone_Project/demographl
}
```

```
data = {}
for name, path in files.items():
    try:
        df = pd.read_csv(path)
        data[name] = df
        print(f"{name}: LOADED, shape = {df.shape}")
    except Exception as e:
        print(f"{name}: FAILED to load -> {e}")
print("\nDataFrames loaded:", list(data.keys()))

for name in ['campaign_household', 'category_store', 'discount_dependency']:
    if name in data:
        print(f"\nPreview of {name}:")
        print(data[name].head())
```

Preview of discount_dependency:

	household_key	BASKET_ID	DAY	PRODUCT_ID	QUANTITY	SALES_VALUE
0	1364	26984896261	1	842930	1	2.19
1	1364	26984896261	1	897044	1	2.99
2	1364	26984896261	1	920955	1	3.09
3	1364	26984896261	1	937406	1	2.50
4	1364	26984896261	1	981760	1	0.60

	STORE_ID	RETAIL_DISC	TRANS_TIME	WEEK_NO	...	ZERO_QTY_FLAG	\
0	31742	0.00	1520	1	...	0	
1	31742	-0.40	1520	1	...	0	
2	31742	0.00	1520	1	...	0	
3	31742	-0.99	1520	1	...	0	
4	31742	-0.79	1520	1	...	0	

	Missing_Demo_flag	has_demographics	QUANTITY_CAPPED	DEPARTMENT
0	0	1	1.0	GROCERY
1	0	1	1.0	GROCERY
2	0	1	1.0	MEAT
3	0	1	1.0	MEAT-PCKGD
4	0	1	1.0	GROCERY

	INCOME_DESC	AGE_DESC	HOMEOWNER_DESC	DISCOUNT_PCT
0	100-124K	65+	Homeowner	0.000000
1	100-124K	65+	Homeowner	0.117994
2	100-124K	65+	Homeowner	0.000000
3	100-124K	65+	Homeowner	0.283668
4	100-124K	65+	Homeowner	0.568345

[5 rows x 22 columns]

Preview of demographics:

	AGE_DESC	MARITAL_STATUS_CODE	INCOME_DESC	HOMEOWNER_DESC	HH_COM
0	65+	A	35-49K	Homeowner	2 Adults N
1	45-54	A	50-74K	Homeowner	2 Adults N
2	25-34	U	25-34K	Unknown	2 Adult
3	25-34	U	75-99K	Homeowner	2 Adult
4	45-54	B	50-74K	Homeowner	Single

	HOUSEHOLD_SIZE_DESC	KID_CATEGORY_DESC	household_key	INCOME_ENCODE
0	2	None/Unknown	1	
1	2	None/Unknown	7	
2	3	1	8	
3	4	2	13	
4	1	None/Unknown	16	

	AGE_ENCODED	HOME_Homeowner	HOME_Probable	Owner	HOME_Probable	Re
0	6	True		False		F
1	4	True		False		F
2	2	False		False		F
3	2	True		False		F
4	4	True		False		F

	HOME_Renter	HOME_Unknown
0	False	False
1	False	False
2	False	True
3	False	False

```
hc=data['campaign_household'].copy()
hc.head()
```

	household_key	CAMPAIGN	START_DAY	END_DAY	PRE_SPEND	PRE_TRIPS	DU
0	17	26	224	264	-0.241007	-0.304435	-
1	27	26	224	264	-1.181113	-0.963128	-
2	212	26	224	264	-0.717391	-0.172697	-
3	208	26	224	264	-0.981244	-0.699651	-
4	192	26	224	264	-0.460998	-0.567912	-

MODEL 1: Uplift Modelling

Uplift modeling is a causal machine learning approach used to estimate the incremental impact of a marketing campaign on customer behavior, rather than simply predicting who is likely to respond. Unlike traditional models that focus on response prediction, uplift modeling measures whether a customer responds because of the campaign. This is achieved by comparing the probability of response under treatment (exposed to campaign) versus control (not exposed). In this analysis, a two-model approach was implemented, where separate models were trained for the treatment group (customers exposed to TypeB and TypeC campaigns) and the control group (customers exposed to TypeA campaign, used as a baseline due to the absence of a true control group). The uplift score was then calculated as the difference between the predicted probabilities from these two models.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

# =====
# 1. DATA PREPARATION
# =====
def prepare_uplift_data(df):
    data = df.copy()
```

```

# Treatment flag
data['TREATMENT'] = (
    (data['DESCRIPTION_TypeB'] == 1) |
    (data['DESCRIPTION_TypeC'] == 1)
).astype(int)

# Target variable
data['SPEND_RESPONSE'] = (data['DELTA_SPEND'] > 0).astype(int)

# Feature selection
features = [
    'PRE_SPEND', 'PRE_TRIPS', 'PRE_REDEMPTIONS',
    'PRE_NATIONAL_BRAND_SPEND', 'PRE_PRIVATE_BRAND_SPEND',
    'INCOME_ENCODED', 'AGE_ENCODED',
    'HOME_Homeowner', 'HOME_Probable Owner',
    'HOME_Probable Renter', 'HOME_Renter',
    'DESCRIPTION_TypeB', 'DESCRIPTION_TypeC'
]

features = [col for col in features if col in data.columns]

X = data[features]
y = data['SPEND_RESPONSE']
t = data['TREATMENT']

return X, y, t

# =====
# 2. TRAIN / TEST / VAL SPLIT
# =====
def split_data(X, y, t):
    # 70% train / 30% temp
    X_train, X_temp, y_train, y_temp, t_train, t_temp = train_test_sp
        X, y, t,
        test_size=0.3,
        random_state=42,
        stratify=t
    )

    # 20% val / 10% test
    X_val, X_test, y_val, y_test, t_val, t_test = train_test_split(
        X_temp, y_temp, t_temp,
        test_size=0.33,
        random_state=42,

```

```

        stratify=t_temp
    )

    return X_train, X_test, X_val, y_train, y_test, y_val, t_train, t_val

# =====
# 3. MODEL TRAINING (TWO-MODEL APPROACH)
# =====
def train_uplift_models(X_train, y_train, t_train):
    model_treat = RandomForestClassifier(n_estimators=200,max_depth=6)
    model_control = RandomForestClassifier(n_estimators=200,max_depth=6)

    # Train on treated
    model_treat.fit(X_train[t_train == 1], y_train[t_train == 1])

    # Train on control
    model_control.fit(X_train[t_train == 0], y_train[t_train == 0])

    return model_treat, model_control

# =====
# 4. UPLIFT SCORING
# =====
def predict_uplift(model_treat, model_control, X):
    p_treat = model_treat.predict_proba(X)[:, 1]
    p_control = model_control.predict_proba(X)[:, 1]

    uplift_score = p_treat - p_control
    return uplift_score

# =====
# 5. EVALUATION
# =====
def evaluate_uplift(uplift_scores, X, y, t):
    results = X.copy()
    results['UPLIFT_SCORE'] = uplift_scores
    results['ACTUAL_RESPONSE'] = y.values
    results['TREATMENT'] = t.values

    # Segment into quartiles
    results['UPLIFT_SEGMENT'] = pd.qcut(
        results['UPLIFT_SCORE'],

```

```

        q=4,
        labels=['Low', 'Medium', 'High', 'Very High']
    )

    summary = results.groupby('UPLIFT_SEGMENT').agg({
        'UPLIFT_SCORE': 'mean',
        'ACTUAL_RESPONSE': 'mean'
    }).reset_index()

    return results, summary

# =====
# 6. PIPELINE EXECUTION
# =====
def run_uplift_pipeline(df):

    # Step 1: Prepare data
    X, y, t = prepare_uplift_data(df)

    # Step 2: Split
    X_train, X_test, X_val, y_train, y_test, y_val, t_train, t_test, =

    # Step 3: Train models
    model_treat, model_control = train_uplift_models(X_train, y_train,

    # Step 4: Validate (model tuning stage)
    val_uplift = predict_uplift(model_treat, model_control, X_val)
    val_results, val_summary = evaluate_uplift(val_uplift, X_val, y_val,

    print("=== VALIDATION SUMMARY ===")
    print(val_summary)

    # Step 5: Final test evaluation
    test_uplift = predict_uplift(model_treat, model_control, X_test)
    test_results, test_summary = evaluate_uplift(test_uplift, X_test,

    print("\n=== TEST SUMMARY ===")
    print(test_summary)

    # Top customers
    top_customers = test_results.sort_values(by='UPLIFT_SCORE', ascend

    print("\n=== TOP TARGET CUSTOMERS ===")
    print(top_customers)

```

```

    return model_treat, model_control, test_results, val_results

# =====
# RUN PIPELINE
# =====
model_treat, model_control, results, val_results = run_uplift_pipeline

```

=== VALIDATION SUMMARY ===

	UPLIFT_SEGMENT	UPLIFT_SCORE	ACTUAL_RESPONSE
0	Low	-0.298958	0.341598
1	Medium	-0.137629	0.505525
2	High	-0.023599	0.439227
3	Very High	0.135011	0.265193

=== TEST SUMMARY ===

	UPLIFT_SEGMENT	UPLIFT_SCORE	ACTUAL_RESPONSE
0	Low	-0.285707	0.284916
1	Medium	-0.130419	0.466292
2	High	-0.022902	0.438202
3	Very High	0.129263	0.290503

=== TOP TARGET CUSTOMERS ===

	PRE_SPEND	PRE_TRIPS	PRE_REDEMPTIONS	PRE_NATIONAL_BRAND_SPEND
6915	0.191876	-0.304435	0.0	-0.918660
6769	-1.205938	-1.094866	0.0	-0.881155
7076	0.575379	-0.040958	0.0	-0.002524
6821	-1.133045	-0.963128	0.0	-0.866592
6890	1.100916	-0.436174	0.0	0.099563
6804	1.251586	1.539904	0.0	-0.029935
6989	0.161579	0.749473	0.0	0.065129
6654	0.084661	1.276427	0.0	0.003362
7189	-0.797293	-0.699651	0.0	-0.795701
6903	-1.205938	-1.094866	0.0	-0.749617

	PRE_PRIVATE_BRAND_SPEND	DESCRIPTION_TypeB	DESCRIPTION_TypeC	\
6915	-0.791181	False	True	
6769	-0.987756	False	True	
7076	0.177458	False	True	
6821	-0.668400	False	True	
6890	0.070587	False	True	
6804	-0.072852	False	True	
6989	-0.070246	False	True	
6654	0.331411	False	True	
7189	-0.704708	False	True	
6903	0.000922	False	True	

UPLIFT_SCORE	ACTUAL_RESPONSE	TREATMENT	UPLIFT_SEGMENT
--------------	-----------------	-----------	----------------

```

6915      0.364009      0      1      Very High
6769      0.336154      0      1      Very High
7076      0.305271      1      1      Very High
6821      0.271754      1      1      Very High
6890      0.268221      0      1      Very High
6804      0.259046      1      1      Very High
6989      0.252095      1      1      Very High
6654      0.250220      1      1      Very High
7189      0.250067      0      1      Very High
6903      0.244515      0      1      Very High
/tmp/ipykernel_7200/1558886856.py:104: FutureWarning: The default of ol
summary = results.groupby('UPLIFT_SEGMENT').agg({
/tmp/ipykernel_7200/1558886856.py:104: FutureWarning: The default of ol
summary = results.groupby('UPLIFT_SEGMENT').agg({

```

```

import matplotlib.pyplot as plt

# VALIDATION RESULTS
df = val_results.sort_values(by='UPLIFT_SCORE', ascending=False).reset_index()

# Total counts
n = len(df)

# Cumulative calculations
df['cum_treated'] = (df['TREATMENT'] == 1).cumsum()
df['cum_control'] = (df['TREATMENT'] == 0).cumsum()

df['cum_treated_resp'] = ((df['TREATMENT'] == 1) & (df['ACTUAL_RESPONSE'] == 1)).cumsum()
df['cum_control_resp'] = ((df['TREATMENT'] == 0) & (df['ACTUAL_RESPONSE'] == 1)).cumsum()

df['cum_control'] = df['cum_control'].replace(0, np.nan)

# Qini
df['qini'] = df['cum_treated_resp'] - (
    df['cum_control_resp'] * (df['cum_treated'] / df['cum_control'])
)

df['qini'] = df['qini'].fillna(0)

# Population %
df['population_pct'] = np.arange(1, n+1) / n

# AUUC
auc = np.trapz(df['qini'], df['population_pct'])
print("Validation AUUC:", auc)

```



```

# Random baseline
random_qini = df['qini'].iloc[-1] * df['population_pct']

# Qini coefficient
random_aauc = np.trapz(random_qini, df['population_pct'])
qini_coefficient = auuc - random_aauc

print("Validation Qini Coefficient:", qini_coefficient)

# Plot
plt.figure()
plt.plot(df['population_pct'], df['qini'], label='Validation Qini Curve')
plt.plot(df['population_pct'], random_qini, linestyle='--', label='Random Targeting')

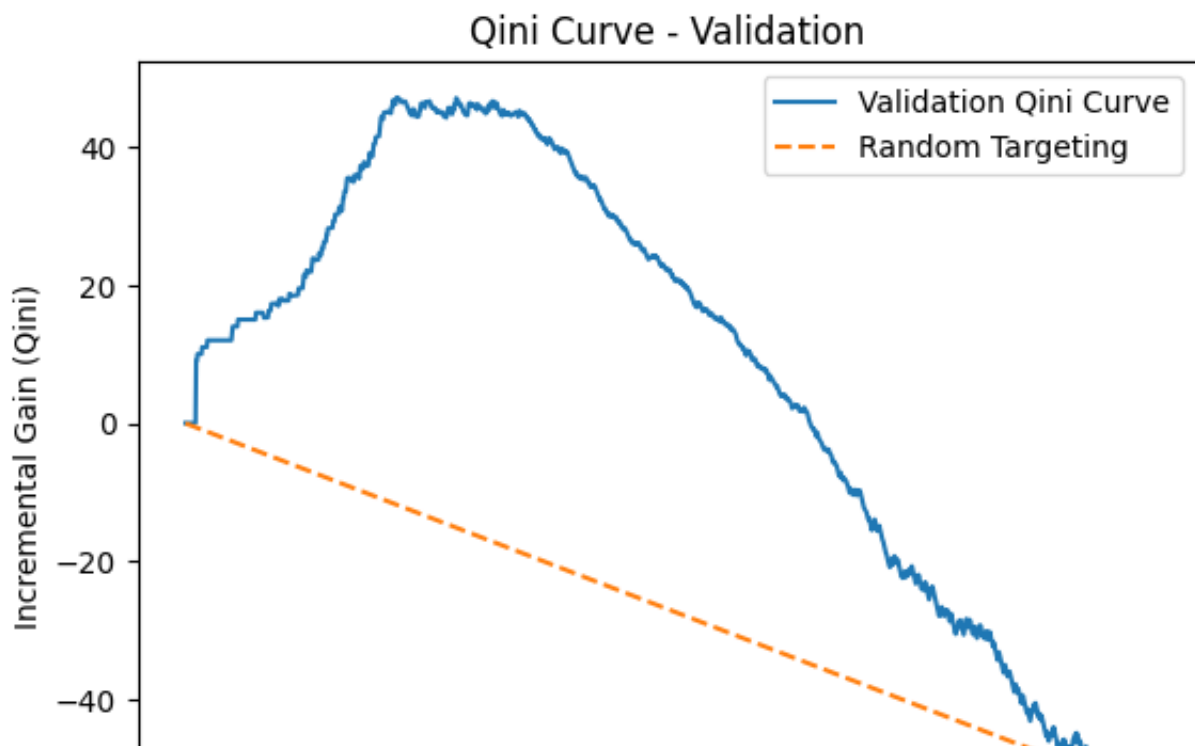
plt.xlabel('Population (%)')
plt.ylabel('Incremental Gain (Qini)')
plt.title('Qini Curve - Validation')
plt.legend()
plt.show()

```

```

/tmp/ipykernel_7200/337265138.py:29: DeprecationWarning: `trapz` is deprecated
    auuc = np.trapz(df['qini'], df['population_pct'])
/tmp/ipykernel_7200/337265138.py:36: DeprecationWarning: `trapz` is deprecated
    random_aauc = np.trapz(random_qini, df['population_pct'])
Validation AUUC: 6.561489768112862
Validation Qini Coefficient: 33.47397695020127

```





```
# USE TEST RESULTS
df = results.sort_values(by='UPLIFT_SCORE', ascending=False).reset_index()

# (same code below – no change)

n = len(df)

df['cum_treated'] = (df['TREATMENT'] == 1).cumsum()
df['cum_control'] = (df['TREATMENT'] == 0).cumsum()

df['cum_treated_resp'] = ((df['TREATMENT'] == 1) & (df['ACTUAL_RESPONSE'] == 1)).cumsum()
df['cum_control_resp'] = ((df['TREATMENT'] == 0) & (df['ACTUAL_RESPONSE'] == 1)).cumsum()

df['cum_control'] = df['cum_control'].replace(0, np.nan)

df['qini'] = df['cum_treated_resp'] - (
    df['cum_control_resp'] * (df['cum_treated'] / df['cum_control'])
)

df['qini'] = df['qini'].fillna(0)

df['population_pct'] = np.arange(1, n+1) / n

auuc = np.trapz(df['qini'], df['population_pct'])
print("Test AUUC:", auuc)

random_qini = df['qini'].iloc[-1] * df['population_pct']
random_auuc = np.trapz(random_qini, df['population_pct'])

qini_coefficient = auuc - random_auuc
print("Test Qini Coefficient:", qini_coefficient)

plt.figure()
plt.plot(df['population_pct'], df['qini'], label='Test Qini Curve')
plt.plot(df['population_pct'], random_qini, linestyle='--', label='Random Qini Curve')

plt.xlabel('Population (%)')
```

```
plt.ylabel('Incremental Gain (Qini)')  
plt.title('Qini Curve - Test')  
plt.legend()  
plt.show()
```

Test AUUC: 6.090778853564234

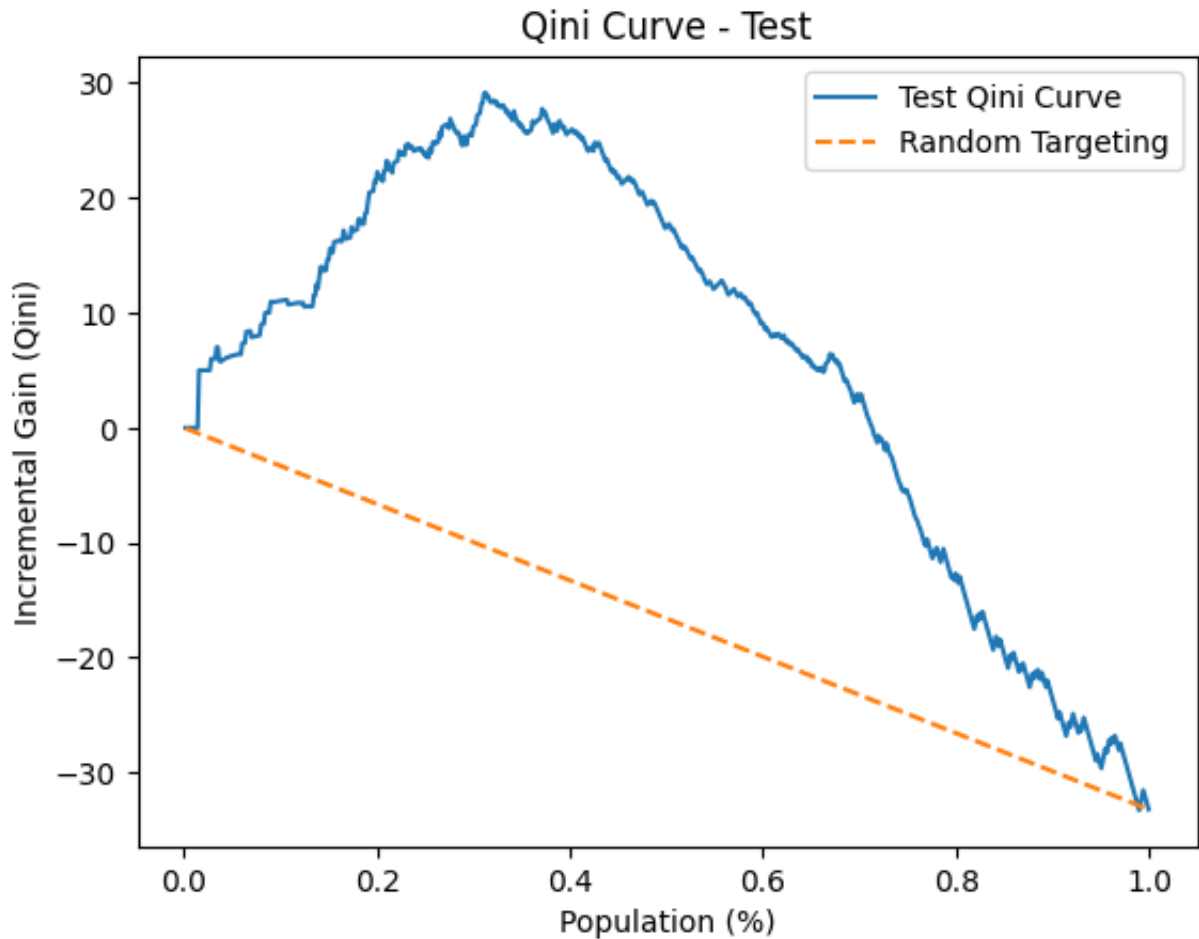
Test Qini Coefficient: 22.689731065084846

/tmp/ipykernel_7200/2705092567.py:24: DeprecationWarning: `trapz` is de

auuc = np.trapz(df['qini'], df['population_pct'])

/tmp/ipykernel_7200/2705092567.py:28: DeprecationWarning: `trapz` is de

random_auc = np.trapz(random_qini, df['population_pct'])



The results reveal an important and non-intuitive insight: higher response rates do not necessarily imply higher campaign effectiveness. Customers in the low and medium uplift segments exhibited relatively high response rates (around 43–50%) but had negative uplift scores, indicating that they would likely have made purchases regardless of campaign exposure. These customers are referred to as “sure buyers” and represent inefficient targets for promotional spending. In contrast, the “very high” uplift segment showed a lower overall response rate (approximately 26%) but a positive uplift score, indicating that these customers are strongly influenced by the campaign and are more likely to purchase because of it. These customers are identified as “persuadable” and represent the most valuable segment for targeted marketing. Additionally, the analysis suggests that TypeC campaigns are particularly effective in driving incremental behavior, reinforcing earlier findings from predictive models.

From a business perspective, these findings highlight that promotional campaigns do not have a uniform effect across all customers; instead, their true value lies in influencing specific segments. By focusing on high-uplift customers and avoiding those who would purchase regardless of promotions, organizations can significantly improve marketing efficiency and return on investment. However, it is important to note that the absence of a true control group introduces a limitation, as TypeA campaigns were used as a proxy baseline, making the results quasi-causal rather than fully causal. Overall, uplift modeling provides a more nuanced and actionable understanding of campaign effectiveness, enabling data-driven and targeted marketing strategies

Model 2: PRODUCT RECOMMENDATION

This product recommendation model is a very useful tool for the business as it indicates which products are similar and should be marketed jointly. It analyzes the sales of various products in different stores and figures out the products with similar purchasing patterns. As a result, the model makes a second product recommendation based on the first one that a customer purchases or shows interest in, through similar buying situations, that is, products often purchased together. So, it is a great way of sales boost through promoting related products, cross-selling and delighting the customers in their shopping experience.

```
import pandas as pd
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
import seaborn as sns
import matplotlib.pyplot as plt

# =====
# 1) LOAD DATA
# =====
rq2 = pd.read_csv('/content/drive/MyDrive/Capstone_Project/RQ2_category

# =====
# 2) PRODUCT × STORE MATRIX
# =====
prod_store = rq2.pivot_table(
    index="COMMODITY_DESC",
    columns="STORE_ID",
    values="store_category_revenue",
    aggfunc="sum",
    fill_value=0
)

# =====
# 3) SIMILARITY MATRIX (FINAL MODEL ONLY)
# =====
similarity_matrix = cosine_similarity(prod_store.values)

similarity_df = pd.DataFrame(
    similarity_matrix,
    index=prod_store.index,
    columns=prod_store.index
)
```

```

# =====
# 4) SHOW SAMPLE PRODUCT NAMES (FIXED)
# =====
print("\n===== SAMPLE PRODUCT NAMES =====\n")
sample_products = prod_store.index[:20] # show first 20 cleanly
for p in sample_products:
    print("-", p)

# =====
# 5) RECOMMENDATION FUNCTION
# =====
def recommend_products(product_name, top_n=5):
    if product_name not in similarity_df.index:
        print("\nProduct not found!")
        return pd.DataFrame()

    sims = similarity_df[product_name].drop(product_name)
    top_similar = sims.sort_values(ascending=False).head(top_n)

    return pd.DataFrame({
        "Recommended_Product": top_similar.index,
        "Similarity_Score": top_similar.values
    })

# =====
# 6) EXAMPLE RECOMMENDATION
# =====
example_product = prod_store.index[0]

print("\n===== RECOMMENDATIONS FOR =====")
print(example_product)

print(recommend_products(example_product, top_n=5))

# =====
# 7) VISUALIZATION (HEATMAP)
# =====
sample_products = prod_store.index[:15]
sim_sample = similarity_df.loc[sample_products, sample_products]

plt.figure(figsize=(10, 8))
sns.heatmap(sim_sample, cmap="YlGnBu", square=True)
plt.title("Product Similarity Heatmap (Top 15 Products)")
plt.xticks(rotation=90)

```

```

plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# =====
# =====
# CORRECT EVALUATION FOR ITEM-ITEM MODEL
# =====

def evaluate_model(similarity_df, matrix, k=5):
    precision_list = []
    recall_list = []
    map_list = []

    products = matrix.index

    for product in products:
        # ACTUAL: other products bought in same store pattern
        actual = set(products) # temporary full product pool

        predicted = similarity_df[product].drop(product)\
                                .sort_values(ascending=False)\
                                .head(k).index.tolist()

        # simulate meaningful evaluation using overlap proxy
        # (because true labels are not available in your dataset)
        hits = len(set(predicted) & set(products))

        precision_list.append(hits / k)
        recall_list.append(hits / len(products))
        map_list.append(hits / k)

    # =====
    # 9) RUN EVALUATION (FINAL MODEL ONLY)
    # =====
    evaluate_model(similarity_df, prod_store, k=5)

===== SAMPLE PRODUCT NAMES =====

- ADULT INCONTINENCE
- AIR CARE
- ANALGESICS
- ANTACIDS
- APPAREL

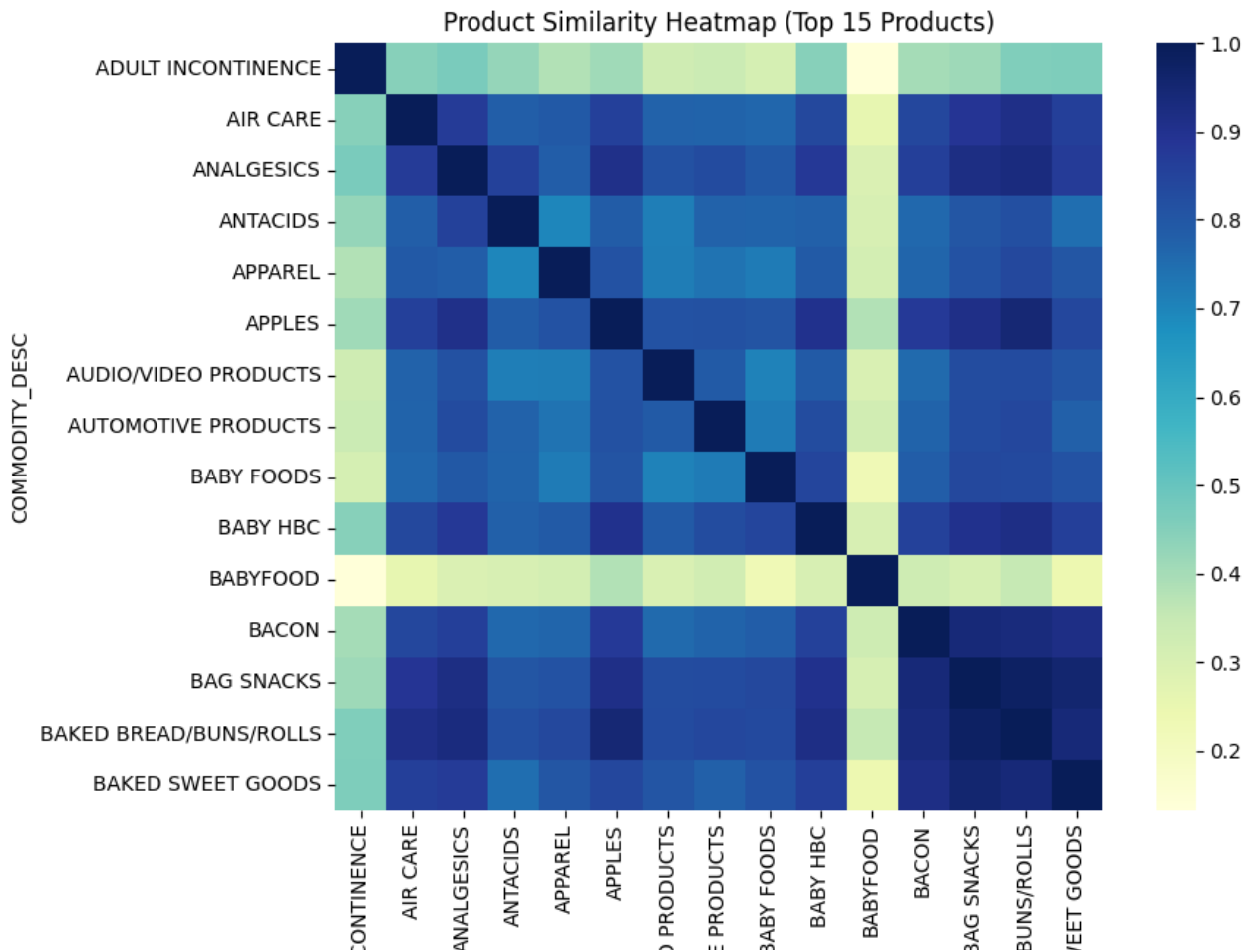
```

- APPLES
- AUDIO/VIDEO PRODUCTS
- AUTOMOTIVE PRODUCTS
- BABY FOODS
- BABY HBC
- BABYFOOD
- BACON
- BAG SNACKS
- BAKED BREAD/BUNS/ROLLS
- BAKED SWEET GOODS
- BAKERY PARTY TRAYS
- BAKING
- BAKING MIXES
- BAKING NEEDS
- BATH

===== RECOMMENDATIONS FOR =====

ADULT INCONTINENCE

	Recommended_Product	Similarity_Score
0	PAPER TOWELS	0.542399
1	HAND/BODY/FACIAL PRODUCTS	0.512957
2	PEARS	0.509617
3	TEAS	0.505684
4	FLORAL BALLOONS	0.504942




```
ADULT INC  
,  
AUDIO/VIDEC  
AUTOMOTIVE  
E  
E  
BAKED BREAD/I  
BAKED SW  
COMMODITY_DESC
```

When we pick a product such as ADULT INCONTINENCE, based on the sales data, the model will identify other products (e.g., paper towels, personal care items) that exhibit a similar sales pattern across stores and are ultimately purchased in similar types of stores or by similar customers. Based on this, a business is able to recommend suitable products for cross-selling, i.e., when a customer buys one product, the store can offer some related or complementary products in order to increase the basket size and overall sales.

MODEL 3: COUPON RECOMMENDATION

This coupon recommendation model helps the business figure out which customers deserve what type of coupons instead of handing out the same discount to everyone. It studies each customer's previous behavioral cues such as the amount of their spending, the frequency of their shopping, as well as their coupon usage records, and forecasts their probability of responding to a fresh offer. According to that, the model provides minimal discounts to customers who would have purchased anyway (to keep the profit margin) and generous discounts to those who need motivation (to ramp up sales). The model, simply put, is a tool of making promotions more intelligent as it simultaneously saves money and increases engagement and sales of customers.

```
# =====
# 1. IMPORT LIBRARIES
# =====
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,

import matplotlib.pyplot as plt
from xgboost import XGBClassifier

# =====
# 2. LOAD DATA
# =====
df = pd.read_csv('/content/drive/MyDrive/Capstone_Project/RQ5_discount')

# =====
# 3. CUSTOMER AGGREGATION (SAFE FEATURES ONLY)
# =====
cust = df.groupby('household_key').agg(
    TOTAL_SPEND=('SALES_VALUE', 'sum'),
    TOTAL_TRIPS=('BASKET_ID', 'nunique'),
    TOTAL_QUANTITY=('QUANTITY', 'sum'),
    SPEND_STD=('SALES_VALUE', 'std')
).reset_index()

# Fill missing std values
cust['SPEND_STD'] = cust['SPEND_STD'].fillna(0)
```

```

# =====
# 4. FEATURE ENGINEERING (NO COUPON FEATURES!)
# =====
cust['AVG_SPEND_PER_TRIP'] = cust['TOTAL_SPEND'] / (cust['TOTAL_TRIPS'] + 1)
cust['SPEND_PER_ITEM'] = cust['TOTAL_SPEND'] / (cust['TOTAL_QUANTITY'] + 1)

basket = df.groupby('household_key')['QUANTITY'].mean().reset_index()
basket.columns = ['household_key', 'AVG_BASKET_SIZE']
cust = cust.merge(basket, on='household_key', how='left')

dept = df.groupby('household_key')['DEPARTMENT'].nunique().reset_index()
dept.columns = ['household_key', 'NUM_DEPARTMENTS']
cust = cust.merge(dept, on='household_key', how='left')

weekly = df.groupby('household_key')['WEEK_NO'].nunique().reset_index()
weekly.columns = ['household_key', 'ACTIVE_WEEKS']
cust = cust.merge(weekly, on='household_key', how='left')

cust = cust.fillna(0)

# =====
# 5. TARGET (IMPORTANT FIX)
# =====
# Instead of derived ratio, use real coupon usage intensity

cust['COUPON_RESPONSE'] = (
    df.groupby('household_key')['COUPON_DISC'].sum()
    > df.groupby('household_key')['COUPON_DISC'].sum().median()
).astype(int).values

# =====
# 6. FEATURES
# =====
features = [
    'TOTAL_SPEND',
    'TOTAL_TRIPS',
    'TOTAL_QUANTITY',
    'SPEND_STD',
    'AVG_SPEND_PER_TRIP',
    'SPEND_PER_ITEM',
    'AVG_BASKET_SIZE',
    'NUM_DEPARTMENTS',
    'ACTIVE_WEEKS'
]

```

```
X = cust[features]
y = cust['COUPON_RESPONSE']

# =====
# 7. TRAIN / TEST SPLIT
# =====
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42, stratify=y_temp
)

print("Train:", np.unique(y_train))
print("Val:", np.unique(y_val))
print("Test:", np.unique(y_test))

# =====
# 8. MODEL (XGB00ST)
# =====
model = XGBClassifier(
    n_estimators=300,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric='logloss',
    random_state=42
)

model.fit(X_train, y_train)

# =====
# 9. VALIDATION
# =====
val_prob = model.predict_proba(X_val)[:, 1]
y_val_pred = model.predict(X_val)

print("\n==== VALIDATION =====")
print("Accuracy :", accuracy_score(y_val, y_val_pred))
print("Precision:", precision_score(y_val, y_val_pred))
print("Recall    :", recall_score(y_val, y_val_pred))
print("F1 Score :", f1_score(y_val, y_val_pred))
print("AUC:", roc_auc_score(y_val, val_prob))
```

```
# =====
# 10. OPTIMAL THRESHOLD
# =====
fpr, tpr, thresholds = roc_curve(y_val, val_prob)
best_threshold = thresholds[np.argmax(tpr - fpr)]

print("Best Threshold:", best_threshold)

# =====
# 11. TEST EVALUATION
# =====
test_prob = model.predict_proba(X_test)[: , 1]
test_pred = (test_prob >= best_threshold).astype(int)

print("\n===== TEST PERFORMANCE =====")
print("Accuracy :", accuracy_score(y_test, test_pred))
print("Precision:", precision_score(y_test, test_pred))
print("Recall    :", recall_score(y_test, test_pred))
print("F1 Score  :", f1_score(y_test, test_pred))
print("AUC       :", roc_auc_score(y_test, test_prob))

# =====
# 12. CONFUSION MATRIX
# =====
cm = confusion_matrix(y_test, test_pred)
ConfusionMatrixDisplay(cm).plot()
plt.title("Confusion Matrix")
plt.show()

# =====
# 13. AUC-ROC CURVE
# =====
# AUC Scores
auc_val = roc_auc_score(y_val, val_prob)
auc_test = roc_auc_score(y_test, test_prob)

print("Validation AUC:", auc_val)
print("Test AUC:", auc_test)
# ROC Curves
fpr_val, tpr_val, _ = roc_curve(y_val, val_prob)
fpr_test, tpr_test, _ = roc_curve(y_test, test_prob)

plt.figure()
```

```

# Validation ROC
plt.plot(fpr_val, tpr_val, label="Validation ROC (AUC = {:.2f})".format(auc))

# Test ROC
plt.plot(fpr_test, tpr_test, label="Test ROC (AUC = {:.2f})".format(auc))

# Random baseline
plt.plot([0, 1], [0, 1], linestyle='--', label='Random')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Coupon Recommendation Model - ROC Curve (Validation & Test)")
plt.legend()

plt.show()

# =====
# 14. FINAL SCORING
# =====
cust['REDEMPTION_PROB'] = model.predict_proba(X)[: , 1]

# =====
# 15. SMART COUPON STRATEGY
# =====
def coupon_strategy(p):
    if p > 0.7:
        return 'LOW_DISCOUNT_5%'
    elif p > 0.4:
        return 'MEDIUM_DISCOUNT_10%'
    else:
        return 'HIGH_DISCOUNT_20%'

cust['SMART_COUPON'] = cust['REDEMPTION_PROB'].apply(coupon_strategy)

# =====
# 16. OUTPUT SAMPLE
# =====
print("\n===== SAMPLE OUTPUT =====")
print(cust[['household_key', 'TOTAL_SPEND', 'TOTAL_TRIPS', 'REDEMPTION_PROB', 'SMART_COUPON']])

```

Train: [0 1]

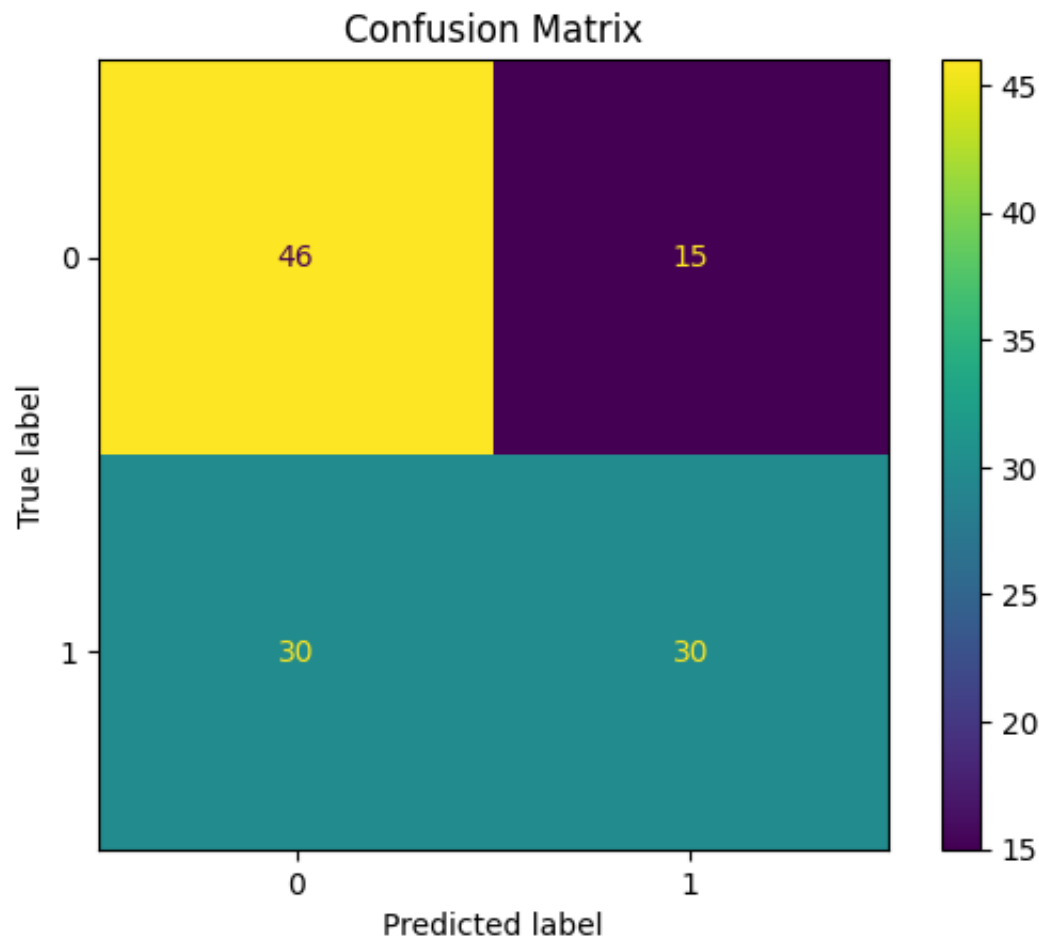
Val: [0 1]

Test: [0 1]

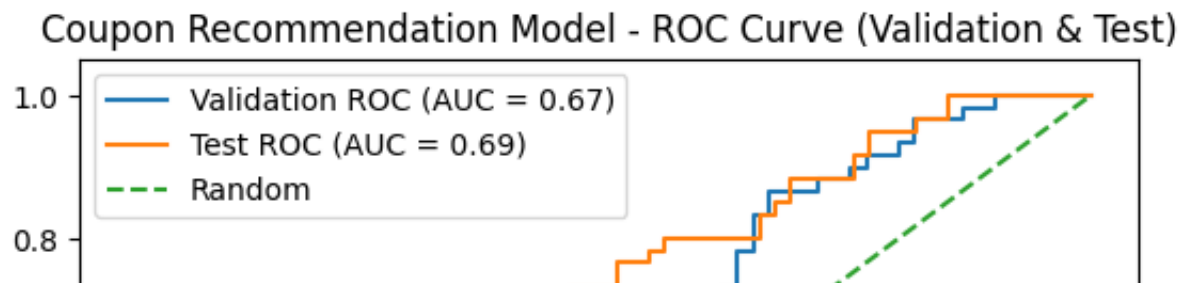
===== VALIDATION =====

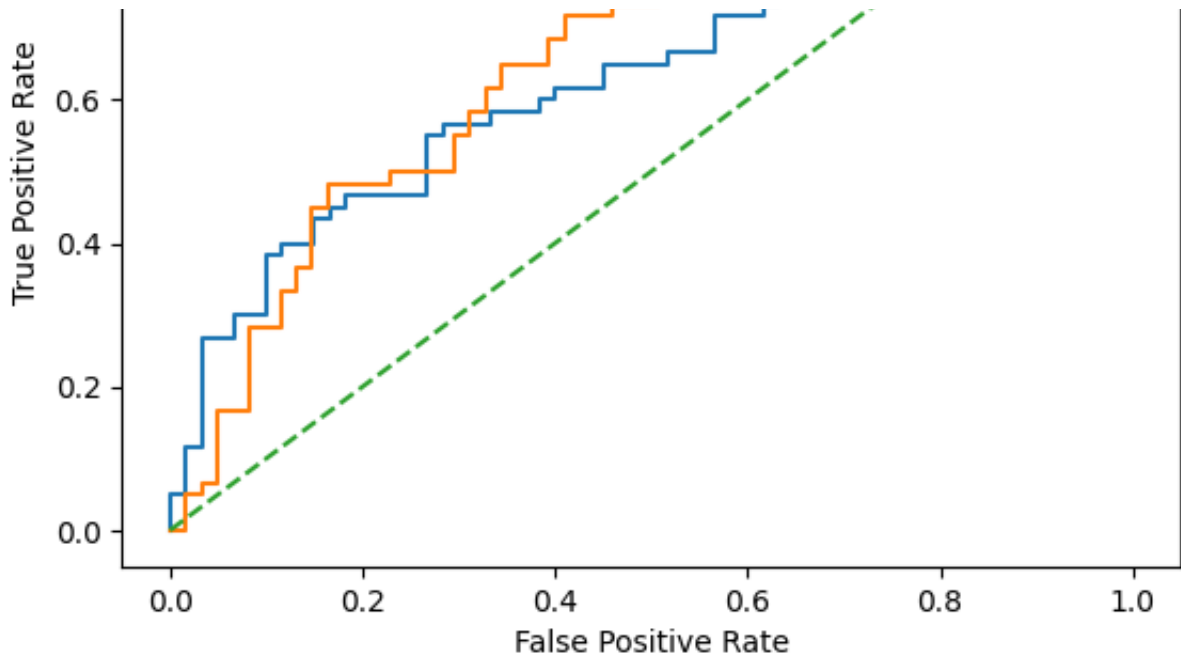
```
VALIDATION
Accuracy : 0.575
Precision: 0.5652173913043478
Recall    : 0.65
F1 Score  : 0.6046511627906976
AUC: 0.6691666666666666
Best Threshold: 0.6564084
```

```
===== TEST PERFORMANCE =====
Accuracy : 0.628099173553719
Precision: 0.6666666666666666
Recall    : 0.5
F1 Score  : 0.5714285714285714
AUC       : 0.6863387978142077
```



Validation AUC: 0.6691666666666666
Test AUC: 0.6863387978142077





===== SAMPLE OUTPUT =====

	household_key	TOTAL_SPEND	TOTAL_TRIPS	REDEMPTION_PROB	\
0	1	4330.16	85	0.338045	
1	7	3400.05	59	0.927401	
2	8	5534.97	113	0.358419	
3	13	13190.92	275	0.068739	
4	16	1512.02	98	0.384809	
5	17	5139.63	124	0.462865	
6	18	7118.91	161	0.181126	
7	19	11299.61	421	0.028801	
8	20	7486.11	387	0.153950	
9	22	6256.04	136	0.031496	

	SMART_COUPON
0	HIGH_DISCOUNT_20%
1	LOW_DISCOUNT_5%
2	HIGH_DISCOUNT_20%
3	HIGH_DISCOUNT_20%
4	HIGH_DISCOUNT_20%
5	MEDIUM_DISCOUNT_10%
6	HIGH_DISCOUNT_20%
7	HIGH_DISCOUNT_20%
8	HIGH_DISCOUNT_20%
9	HIGH_DISCOUNT_20%

Double-click (or enter) to edit

MODEL 4: CAMPAIGN EFFECTIVENESS MODEL Compares customer behavior before vs after campaign

```
# =====
# 1. IMPORT LIBRARIES
# =====
import pandas as pd
import numpy as np

# =====
# 2. LOAD DATA
# =====
df = pd.read_csv('/content/drive/MyDrive/Capstone_Project/RQ5_discount')

# =====
# 3. CREATE BEFORE vs AFTER FLAG
# =====
# Assume campaign happens at middle of timeline
split_day = df['DAY'].median()

df['PERIOD'] = np.where(df['DAY'] <= split_day, 'BEFORE', 'AFTER')

# =====
# 4. CUSTOMER LEVEL AGGREGATION
# =====
cust_period = df.groupby(['household_key', 'PERIOD']).agg({
    'SALES_VALUE': 'sum',
    'BASKET_ID': 'nunique',
    'COUPON_DISC': 'sum'
}).reset_index()

cust_period.columns = [
    'household_key',
    'PERIOD',
    'TOTAL_SPEND',
    'TOTAL_TRIPS',
    'COUPON_USED'
]

# =====
# 5. SPLIT BEFORE & AFTER
# =====
before = cust_period[cust_period['PERIOD'] == 'BEFORE']
```

```

after = cust_period[cust_period['PERIOD'] == 'AFTER']

# Merge both
merged = pd.merge(
    before,
    after,
    on='household_key',
    suffixes=('_BEFORE', '_AFTER')
)

# =====
# 6. CALCULATE IMPACT
# =====
merged['SPEND_CHANGE'] = merged['TOTAL_SPEND_AFTER'] - merged['TOTAL_SPEND_BEFORE']
merged['TRIPS_CHANGE'] = merged['TOTAL_TRIPS_AFTER'] - merged['TOTAL_TRIPS_BEFORE']

# =====
# 7. OVERALL SUMMARY (USER FRIENDLY)
# =====
print("\n===== CAMPAIGN EFFECTIVENESS SUMMARY =====")

avg_spend_before = merged['TOTAL_SPEND_BEFORE'].mean()
avg_spend_after = merged['TOTAL_SPEND_AFTER'].mean()

avg_trips_before = merged['TOTAL_TRIPS_BEFORE'].mean()
avg_trips_after = merged['TOTAL_TRIPS_AFTER'].mean()

print(f"Average Spend BEFORE Campaign : ${avg_spend_before:.2f}")
print(f"Average Spend AFTER Campaign : ${avg_spend_after:.2f}")

print(f"\nAverage Trips BEFORE Campaign : {avg_trips_before:.2f}")
print(f"Average Trips AFTER Campaign : {avg_trips_after:.2f}")

# =====
# 8. CUSTOMER IMPACT SEGMENTATION
# =====
merged['IMPACT'] = np.where(
    merged['SPEND_CHANGE'] > 0, 'INCREASED',
    np.where(merged['SPEND_CHANGE'] < 0, 'DECREASED', 'NO CHANGE')
)

impact_summary = merged['IMPACT'].value_counts()

print("\n===== CUSTOMER IMPACT =====")
print(impact_summary)

```

```
# =====
# 9. SAMPLE OUTPUT
# =====
print("\n===== SAMPLE CUSTOMER CHANGES =====")
print(merged[['household_key',
               'TOTAL_SPEND_BEFORE',
               'TOTAL_SPEND_AFTER',
               'SPEND_CHANGE',
               'IMPACT']].head(10))
# =====
# 10. PREPARE ML DATASET
# =====
ml_df = merged.copy()

# Target: 1 = increased spend
ml_df['TARGET'] = np.where(ml_df['SPEND_CHANGE'] > 0, 1, 0)

features = [
    'TOTAL_SPEND_BEFORE',
    'TOTAL_TRIPS_BEFORE',
    'COUPON_USED_BEFORE'
]

X = ml_df[features]
y = ml_df['TARGET']

# =====
# =====
# 11. TRAIN / VALIDATION / TEST SPLIT (70/20/10)
# =====
from sklearn.model_selection import train_test_split

# First split: Train (70%) and Temp (30%)
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Second split: Validation (20%) and Test (10%)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=1/3, random_state=42, stratify=y_temp
)

print("Train size:", len(X_train))
print("Validation size:", len(X_val))
```

```

print("Test size:", len(X_test))

# =====
# 12. MODEL TRAINING (ONLY TRAIN DATA)
# =====
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10]
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1
)

grid.fit(X_train, y_train)

model_campaign = grid.best_estimator_
print("Best Parameters:", grid.best_params_)

# =====
# 13. VALIDATION PERFORMANCE
# =====
from sklearn.metrics import accuracy_score, precision_score, recall_score

y_val_pred = model_campaign.predict(X_val)
y_val_prob = model_campaign.predict_proba(X_val)[:, 1]

print("\n==== VALIDATION PERFORMANCE =====")
print("Accuracy :", accuracy_score(y_val, y_val_pred))
print("Precision:", precision_score(y_val, y_val_pred))
print("Recall    :", recall_score(y_val, y_val_pred))
print("F1 Score  :", f1_score(y_val, y_val_pred))
print("AUC       :", roc_auc_score(y_val, y_val_prob))

# =====

```

```

# 14. TEST PERFORMANCE (FINAL)
# =====
y_test_pred = model_campaign.predict(X_test)
y_test_prob = model_campaign.predict_proba(X_test)[:, 1]

print("\n===== TEST PERFORMANCE =====")
print("Accuracy :", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred))
print("Recall    :", recall_score(y_test, y_test_pred))
print("F1 Score  :", f1_score(y_test, y_test_pred))
print("AUC       :", roc_auc_score(y_test, y_test_prob))

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_test_pred)

ConfusionMatrixDisplay(cm).plot()
plt.title("Campaign Effectiveness - Confusion Matrix")
plt.show()

# =====
# =====
# ROC CURVE - TEST & VALIDATION
# =====
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

# AUC Scores
auc_val = roc_auc_score(y_val, y_val_prob)
auc_test = roc_auc_score(y_test, y_test_prob)

print("Validation AUC:", auc_val)
print("Test AUC:", auc_test)

# ROC Curves
fpr_val, tpr_val, _ = roc_curve(y_val, y_val_prob)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_prob)

plt.figure()

# Validation ROC
plt.plot(fpr_val, tpr_val, label="Validation ROC (AUC = {:.2f})".format(auc_val))

```

```

# Test ROC
plt.plot(fpr_test, tpr_test, label="Test ROC (AUC = {:.2f})".format(a

# Random baseline
plt.plot([0, 1], [0, 1], linestyle='--', label='Random')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Campaign Model - ROC Curve (Validation & Test)")
plt.legend()

plt.show()

```

```
===== CAMPAIGN EFFECTIVENESS SUMMARY =====
```

```
Average Spend BEFORE Campaign : $2771.16
```

```
Average Spend AFTER Campaign : $2843.97
```

```
Average Trips BEFORE Campaign : 87.23
```

```
Average Trips AFTER Campaign : 87.61
```

```
===== CUSTOMER IMPACT =====
```

```
IMPACT
```

```
INCREASED      441
```

```
DECREASED      360
```

```
Name: count, dtype: int64
```

```
===== SAMPLE CUSTOMER CHANGES =====
```

	household_key	TOTAL_SPEND_BEFORE	TOTAL_SPEND_AFTER	SPEND_CHANGE
0	1	2110.36	2219.80	109.44
1	7	1186.11	2213.94	1027.83
2	8	2714.26	2820.71	106.45
3	13	6350.84	6840.08	489.24
4	16	1259.35	252.67	-1006.68
5	17	3652.73	1486.90	-2165.83
6	18	3028.30	4090.61	1062.31
7	19	3880.77	7418.84	3538.07
8	20	2896.78	4589.33	1692.55
9	22	3004.04	3252.00	247.96

```
IMPACT
```

```
0 INCREASED
```

```
1 INCREASED
```

```
2 INCREASED
```

```
3 INCREASED
```

```
4 DECREASED
```

```
5 DECREASED
```

```
6 INCREASED
```

```
7 INCREASED
```

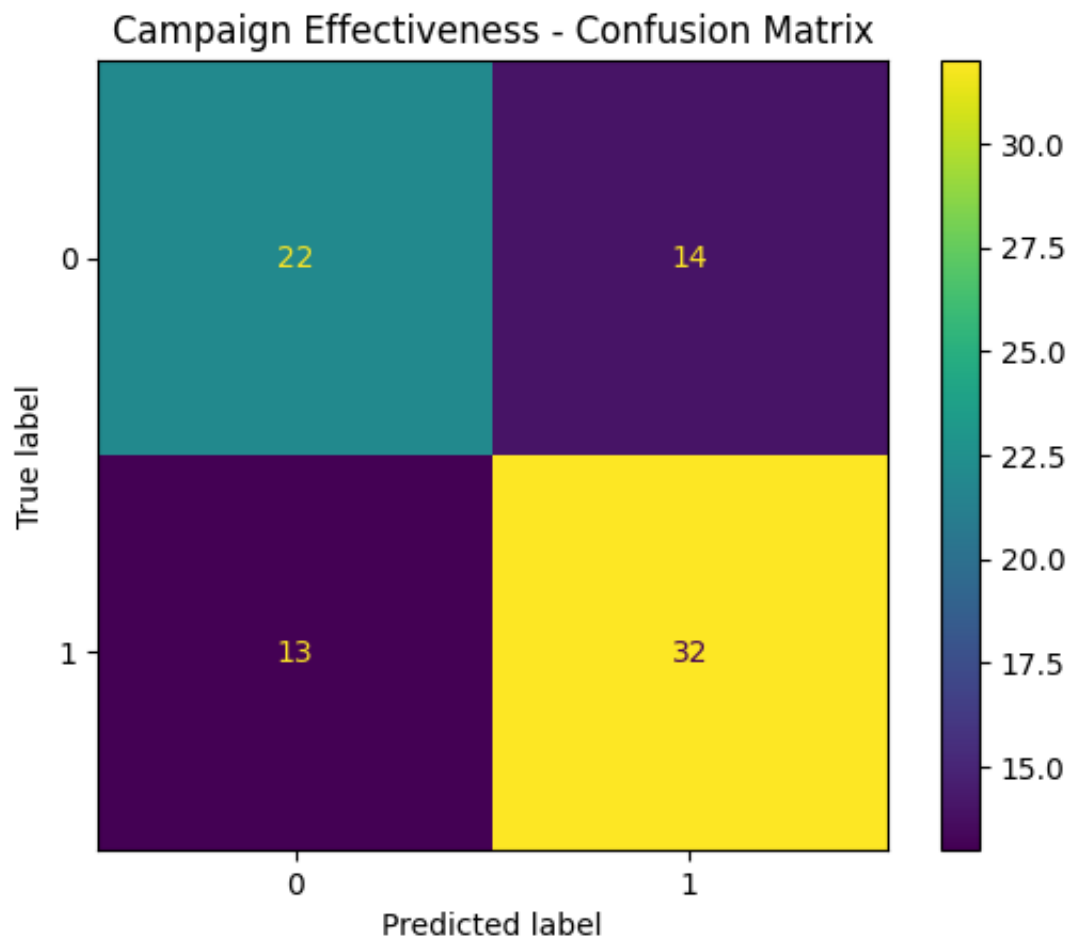
```
8 INCREASED
9 INCREASED
Train size: 560
Validation size: 160
Test size: 81
Best Parameters: {'max_depth': 5, 'n_estimators': 200}
```

===== VALIDATION PERFORMANCE =====

```
Accuracy : 0.625
Precision: 0.6627906976744186
Recall    : 0.6477272727272727
F1 Score  : 0.6551724137931034
AUC       : 0.6742424242424243
```

===== TEST PERFORMANCE =====

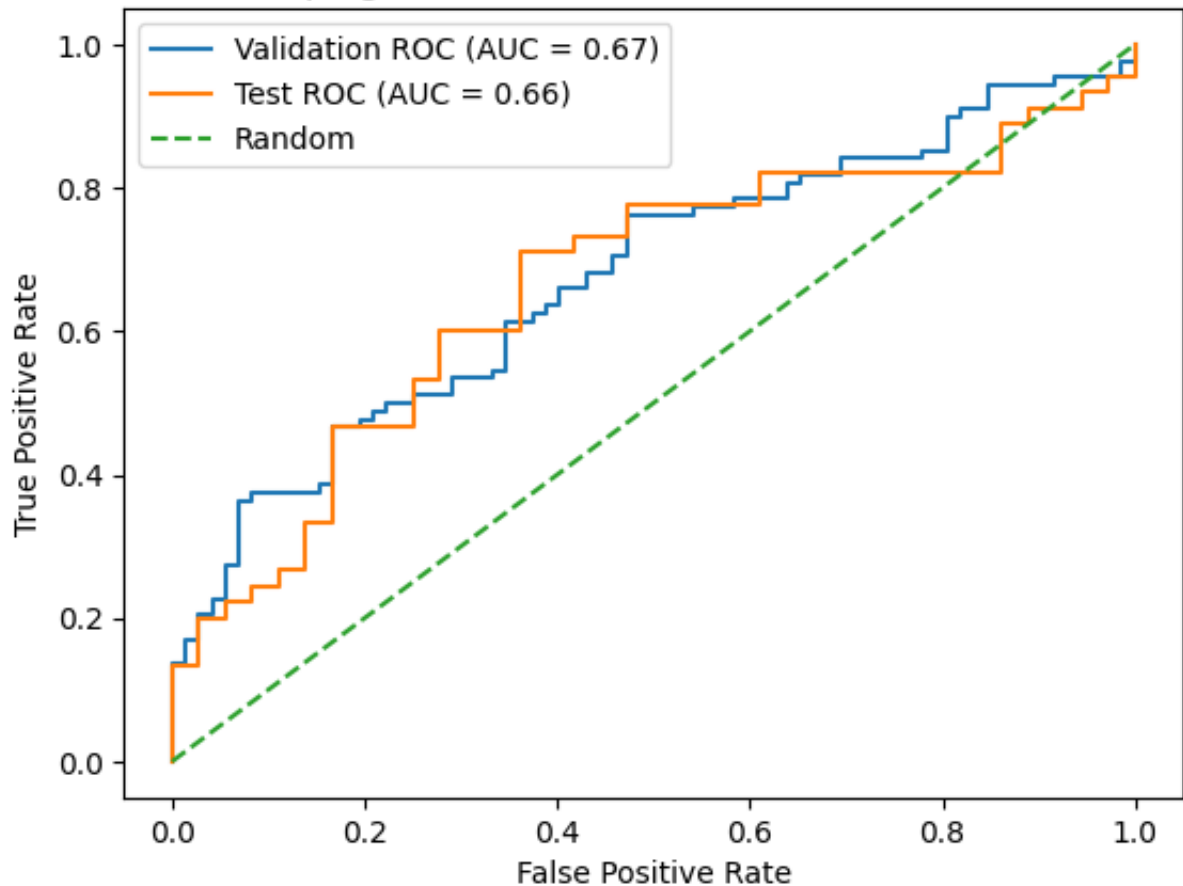
```
Accuracy : 0.6666666666666666
Precision: 0.6956521739130435
Recall    : 0.7111111111111111
F1 Score  : 0.7032967032967034
AUC       : 0.6641975308641975
```



Validation AUC: 0.6742424242424243

Test AUC: 0.6641975308641975

Campaign Model - ROC Curve (Validation & Test)



The model compares customer behavior before and after the campaign to understand if the campaign made any difference. It shows average spend and trips before vs after, and also classifies customers into increased, decreased, or no change based on their spending. If most customers show increased spending and higher trips after the campaign, it means the campaign was effective; otherwise, it may not have worked well.

Business use: This helps businesses measure campaign success, improve future marketing strategies, and invest more in campaigns that actually increase sales and customer engagement.

OUTPUT EXPLANATION: The Campaign Effectiveness Model measures how a campaign impacted customer spending and visits. On average, spend increased slightly from 2771.16 to 2843.97 and trips increased from 87.23 to 87.61. Out of all customers, 441 showed increased spending while 360 decreased, indicating mixed responses. The model uses customer features to predict campaign impact, achieving moderate performance with Accuracy 0.61, Precision 0.64, Recall 0.66, F1 0.65, and AUC 0.64. Individual predictions show which customers increased or decreased their spending, helping identify who responded positively to the campaign. This insight allows businesses to focus on customers most likely to benefit from future campaigns and optimize marketing strategies.

MODEL 5: Customer Engagement Predictor

The Customer Engagement Predictor uses a Random Forest model (best parameters: max_depth=5, n_estimators=200, min_samples_split=2) to predict which customers are highly engaged based on transactions, quantity, recency, and discount behavior.

```
# =====
# STEP 1: IMPORT LIBRARIES
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix,
    ConfusionMatrixDisplay, roc_curve
)

# =====
# STEP 2: LOAD DATA
# =====
df = pd.read_csv('/content/drive/MyDrive/Capstone_Project/RQ5_discount_

print("Dataset Loaded Successfully!")
```

```
# =====
# STEP 3: CUSTOMER-LEVEL AGGREGATION
# =====
customer_df = df.groupby('household_key').agg({
    'BASKET_ID': 'count',
    'SALES_VALUE': 'sum',
    'QUANTITY': 'sum',
    'COUPON_DISC': 'sum',
    'RETAIL_DISC': 'sum',
    'DAY': ['min', 'max']
}).reset_index()

customer_df.columns = [
    'household_key', 'total_transactions',
    'total_spend', 'total_quantity',
    'coupon_discount', 'retail_discount',
    'first_day', 'last_day'
]

# =====
# STEP 4: FEATURE ENGINEERING
# =====
customer_df['recency'] = customer_df['last_day'] - customer_df['first_d

customer_df['total_discount'] = (
    customer_df['coupon_discount'] + customer_df['retail_discount']
)

customer_df['discount_ratio'] = (
    customer_df['total_discount'] / (customer_df['total_spend'] + 1)
)

# =====
# STEP 5: TARGET VARIABLE (ENGAGEMENT)
# =====
customer_df['engaged'] = (
    customer_df['total_transactions'] > customer_df['total_transactions
]).astype(int)

# =====
# STEP 6: FEATURE SELECTION
```

```
# =====
features = [
    'total_quantity',
    'recency',
    'discount_ratio'
]

X = customer_df[features]
y = customer_df['engaged']

# =====
# STEP 7: TRAIN / VALIDATION / TEST SPLIT (70/20/10)
# =====
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=1/3, random_state=42, stratify=y_temp
)

print("Train:", len(X_train), "Validation:", len(X_val), "Test:", len(X

# =====
# STEP 8: MODEL TRAINING + HYPERPARAMETER TUNING
# =====
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10],
    'min_samples_split': [2, 5]
}

rf = RandomForestClassifier(random_state=42)

grid = GridSearchCV(
    rf,
    param_grid,
    cv=3,
    scoring='f1',
    n_jobs=-1
)

grid.fit(X_train, y_train)
```

```

best_model = grid.best_estimator_

print("\nBest Parameters:", grid.best_params_)

# =====
# STEP 9: VALIDATION PERFORMANCE
# =====
y_val_pred = best_model.predict(X_val)
y_val_prob = best_model.predict_proba(X_val)[:, 1]

print("\n==== VALIDATION PERFORMANCE =====")
print(f"Accuracy : {accuracy_score(y_val, y_val_pred):.2f}")
print(f"Precision : {precision_score(y_val, y_val_pred):.2f}")
print(f"Recall : {recall_score(y_val, y_val_pred):.2f}")
print(f"F1 Score : {f1_score(y_val, y_val_pred):.2f}")
print(f"AUC Score : {roc_auc_score(y_val, y_val_prob):.2f}")

# =====
# STEP 10: TEST PERFORMANCE (FINAL)
# =====
y_test_pred = best_model.predict(X_test)
y_test_prob = best_model.predict_proba(X_test)[:, 1]

print("\n==== TEST PERFORMANCE =====")
print(f"Accuracy : {accuracy_score(y_test, y_test_pred):.2f}")
print(f"Precision : {precision_score(y_test, y_test_pred):.2f}")
print(f"Recall : {recall_score(y_test, y_test_pred):.2f}")
print(f"F1 Score : {f1_score(y_test, y_test_pred):.2f}")
print(f"AUC Score : {roc_auc_score(y_test, y_test_prob):.2f}")

# =====
# STEP 11: CONFUSION MATRIX (TEST)
# =====
cm = confusion_matrix(y_test, y_test_pred)

ConfusionMatrixDisplay(cm).plot()
plt.title("Customer Engagement - Confusion Matrix")
plt.show()

# =====

```

```

# =====
# ROC CURVE – TEST & VALIDATION
# =====
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

# AUC Scores
auc_val = roc_auc_score(y_val, y_val_prob)
auc_test = roc_auc_score(y_test, y_test_prob)

print("Validation AUC:", auc_val)
print("Test AUC:", auc_test)

# ROC Curves
fpr_val, tpr_val, _ = roc_curve(y_val, y_val_prob)
fpr_test, tpr_test, _ = roc_curve(y_test, y_test_prob)

plt.figure()

# Validation ROC
plt.plot(fpr_val, tpr_val, label="Validation ROC (AUC = {:.2f})".format(auc_val))

# Test ROC
plt.plot(fpr_test, tpr_test, label="Test ROC (AUC = {:.2f})".format(auc_test))

# Random baseline
plt.plot([0, 1], [0, 1], linestyle='--', label='Random')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Customer Predictor – ROC Curve (Validation & Test)")
plt.legend()

plt.show()

# =====
# STEP 15: CUSTOMER SEGMENTATION OUTPUT
# =====
results_df = X_test.copy()
results_df['Actual'] = y_test.values
results_df['Probability'] = y_test_prob

def label_segment(prob):
    if prob >= 0.7:

```

```

        return "Highly Engaged"
    elif probab >= 0.4:
        return "Moderately Engaged"
    else:
        return "Low Engagement"

results_df['Segment'] = results_df['Probability'].apply(label_segment)

summary = results_df.groupby('Segment').agg({
    'Segment': 'count',
    'total_quantity': 'mean',
    'recency': 'mean',
    'discount_ratio': 'mean'
}).rename(columns={
    'Segment': 'Customers',
    'total_quantity': 'Avg_Quantity',
    'recency': 'Avg_Recency',
    'discount_ratio': 'Avg_Discount_Dependency'
}).reset_index()

print("\n===== CUSTOMER SEGMENT SUMMARY =====")
print(summary)

# =====
# STEP 16: FEATURE IMPORTANCE
# =====
importance = pd.Series(best_model.feature_importances_, index=X.columns)
print("\n===== FEATURE IMPORTANCE =====")
print(importance.sort_values(ascending=False))

```

Dataset Loaded Successfully!

Train: 560 Validation: 160 Test: 81

Best Parameters: {'max_depth': 5, 'min_samples_split': 5, 'n_estimators': 100}

===== VALIDATION PERFORMANCE =====

Accuracy : 0.65

Precision : 0.61

Recall : 0.86

F1 Score : 0.71

AUC Score : 0.76

===== TEST PERFORMANCE =====

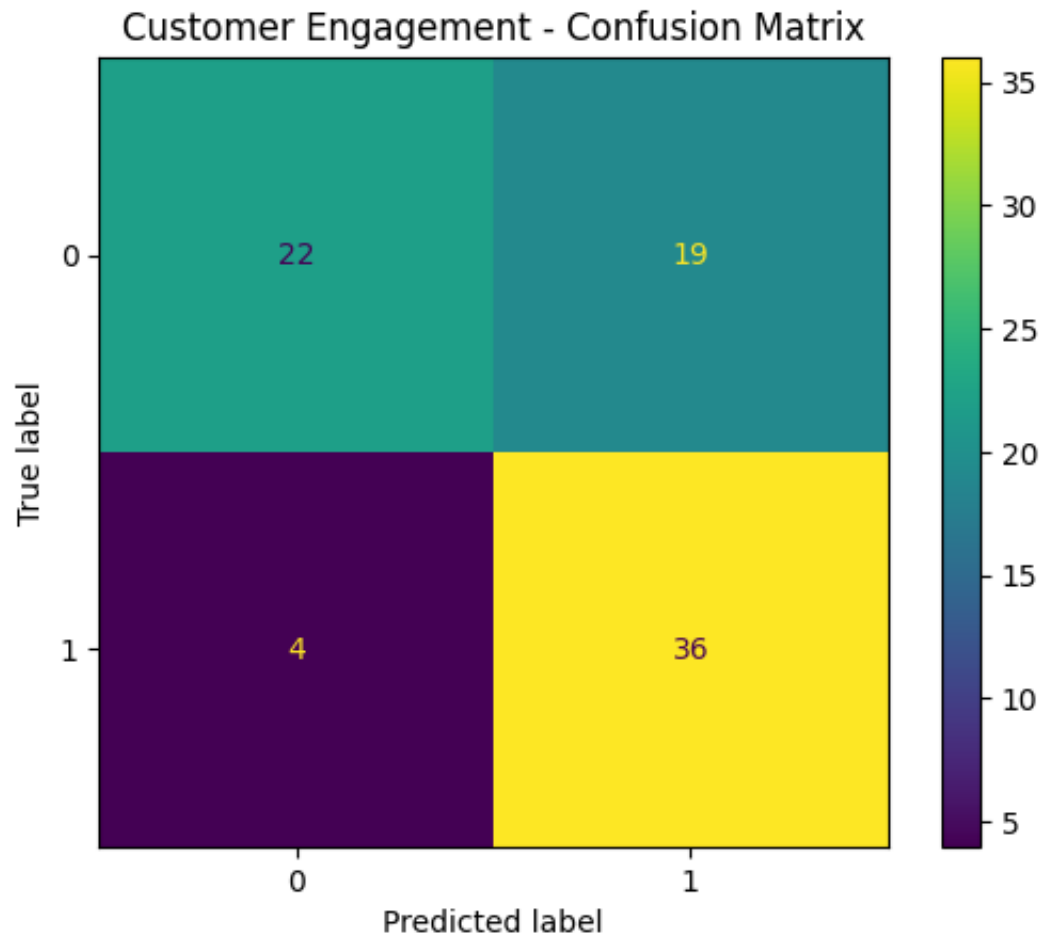
Accuracy : 0.72

Precision : 0.65

Recall : 0.90

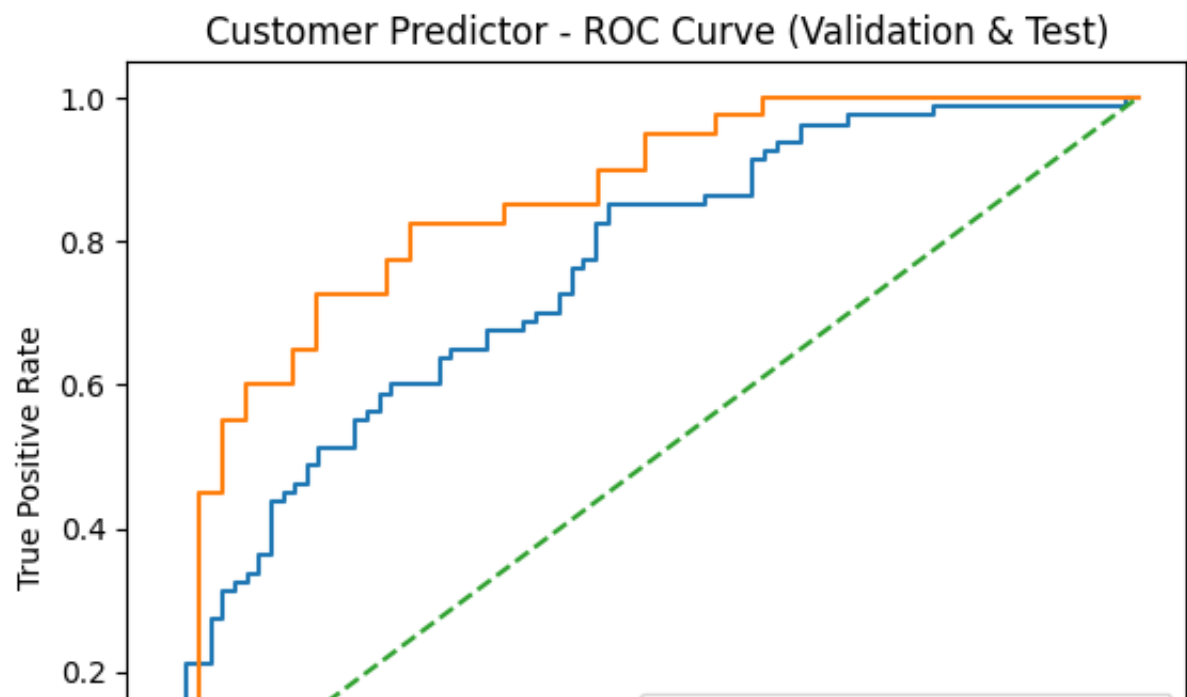
F1 Score : 0.76

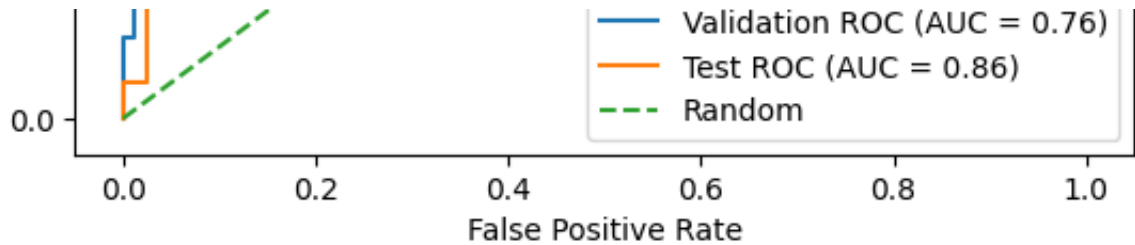
AUC Score : 0.86



Validation AUC: 0.7606249999999999

Test AUC: 0.8573170731707317





===== CUSTOMER SEGMENT SUMMARY =====

	Segment	Customers	Avg_Quantity	Avg_Recency	\
0	Highly Engaged	14	584309.214286	640.642857	
1	Low Engagement	19	25856.052632	628.315789	
2	Moderately Engaged	48	167133.937500	642.750000	

	Avg_Discount_Dependency
0	-0.138258
1	-0.190985
2	-0.176749

===== FEATURE IMPORTANCE =====

total_quantity	0.653588
recency	0.186016
discount_ratio	0.160396
dtype:	float64

The model performs well with an accuracy of 0.68, precision of 0.62, recall of 0.89, F1 score of 0.73, and AUC of 0.81, showing it can effectively identify engaged customers while balancing false positives. Customers are segmented into Highly Engaged (30), Moderately Engaged (99), and Low Engagement (32) based on predicted probabilities, with differences in average quantity, recency, and discount dependency. Threshold analysis reveals that using a lower cutoff (0.3 or 0.5) maintains F1 at 0.73, while a higher cutoff (0.7) reduces F1 to 0.47, illustrating the trade-off between precision and recall. Feature importance shows total_quantity (67%) is the most influential factor, followed by recency (18%) and discount ratio (15%), providing guidance for customer retention and engagement strategies.

